Programming Fundamentals – Pointer Assignment



Subject Title: Pointer Assignment
Subject Name: Programming Fundamentals Lab
Submitted by: Nabiha Shaukat
Roll Number: 26k-3166
Submitted to: Sir Shayan Ali
Date of Submission: 10 Nov 2026

Project Title: Bank Management System Using Pointers in C

Scenario

A bank wants to develop a simple Bank Management System in C to manage customer balances, branch transactions, and banking operations.

The bank maintains the account balances of five customers in an integer array named balance, containing {15000, 25000, 18000, 32000, 12000}. The system should use pointers to access and update customer balances through pointer dereferencing and display their memory addresses.

The bank has two branches, each maintaining monthly transaction records for three months. These records are stored in a 2D array named transactions, containing {{120000, 150000, 180000}, {200000, 170000, 220000}}. The system should use a pointer to an array of three integers and pointer arithmetic to access each branch's records, calculate transaction totals, and display row addresses.

Each branch also maintains a separate customer account array. Branch 1 has balances {15000, 23000, 32000}, while Branch 2 has {12000, 21000, 27000, 35000}. Since both arrays have different sizes, an array of pointers should be used to access customer balances, display their values and addresses, and calculate each branch's total balance.

To avoid repeating calculations, the bank requires a user-defined function named calculateTotal() that receives an account array through a pointer and its size, then returns the total balance.

The system must also support deposits and withdrawals through two functions named deposit() and withdraw(). Both functions accept two integers and return the updated balance. Function pointers should be used to call these banking operations. Furthermore, a function named processTransaction() should receive two integers and a function pointer as arguments, execute the selected operation, and return the result.

The system should demonstrate the practical use of pointers, pointer arithmetic, pointers to 2D arrays, arrays of pointers, passing arrays to functions, and function pointers in a banking environment.

Programming Tasks

Task 1 (Basic Pointers and Account Balance Management):

Create an array named balance containing {15000, 25000, 18000, 32000, 12000}. Use a pointer to access a selected customer's balance, display its value and memory address, and update the balance through pointer dereferencing.

```c
#include <stdio.h>
int main(){
    int balance[5] = {15000, 25000, 18000, 32000, 12000};
    int *ptr;
    int choice;

    printf("Enter customer number (1 to 5): ");
    scanf("%d", &choice);

    ptr = &balance[choice - 1];

    printf("\nCustomer %d balance: %d\n", choice, *ptr);
    printf("Memory address: %p\n", (void*)ptr);

    *ptr = *ptr + 5000;

    printf("\nAfter update:\n");
    printf("New balance: %d\n", *ptr);
    printf("Same value using array: %d\n", balance[choice - 1]);

    return 0;
}
```

```
Enter customer number (1 to 5): 4

Customer 4 balance: 32000
Memory address: 0x7ffe8ada233c

After update:
New balance: 37000
Same value using array: 37000
```

Task 2 (Pointer to a 2D Array):

Create the transactions array containing {{120000, 150000, 180000}, {200000, 170000, 220000}}. Use a pointer to an array of three integers and pointer arithmetic to access both branches' records. Display the transaction values, row addresses, and total transactions of each branch.

```c
#include <stdio.h>
int main(){
    int transactions[2][3] = {{120000, 150000, 180000}, {200000, 170000, 220000}};

    int (*ptr)[3];
    int total1, total2;

    ptr = transactions;
    printf("Branch 1 transactions:\n");
    printf("%d  %d  %d\n", *(*(ptr + 0) + 0), *(*(ptr + 0) + 1), *(*(ptr + 0) + 2));
    printf("Row address: %p\n", (void*)(ptr + 0));

    total1 = *(*(ptr + 0) + 0) + *(*(ptr + 0) + 1) + *(*(ptr + 0) + 2);
    printf("Total of Branch 1: %d\n", total1);

    printf("\nBranch 2 transactions:\n");
    printf("%d  %d  %d\n", *(*(ptr + 1) + 0), *(*(ptr + 1) + 1), *(*(ptr + 1) + 2));
    printf("Row address: %p\n", (void*)(ptr + 1));

    total2 = *(*(ptr + 1) + 0) + *(*(ptr + 1) + 1) + *(*(ptr + 1) + 2);
    printf("Total of Branch 2: %d\n", total2);

    return 0;
}
```

```
Branch 1 transactions:
120000  150000  180000
Row address: 0x7ffc00167590
Total of Branch 1: 450000

Branch 2 transactions:
200000  170000  220000
Row address: 0x7ffc0016759c
Total of Branch 2: 590000
```

Task 3 (Array of Pointers):

Create two customer account arrays: Branch 1 {15000, 23000, 32000} and Branch 2 {12000, 21000, 27000, 35000}. Use an array of pointers to access their balances, display values and memory addresses, and calculate the total balance of each branch.

```c
#include <stdio.h>
int main(){
    int b1[3] = {15000, 23000, 32000};
    int b2[4] = {12000, 21000, 27000, 35000};

    int *branch[2];
    int total1, total2;

    branch[0] = b1;
    branch[1] = b2;

    printf("Branch 1:\n");
    printf("Value: %d  Address: %p\n", *(branch[0] + 0), (void*)(branch[0] + 0));
    printf("Value: %d  Address: %p\n", *(branch[0] + 1), (void*)(branch[0] + 1));
    printf("Value: %d  Address: %p\n", *(branch[0] + 2), (void*)(branch[0] + 2));

    total1 = *(branch[0] + 0) + *(branch[0] + 1) + *(branch[0] + 2);
    printf("Total of Branch 1: %d\n", total1);

    printf("\nBranch 2:\n");
    printf("Value: %d   Address: %p\n", *(branch[1] + 0), (void*)(branch[1] + 0));
    printf("Value: %d   Address: %p\n", *(branch[1] + 1), (void*)(branch[1] + 1));
    printf("Value: %d   Address: %p\n", *(branch[1] + 2), (void*)(branch[1] + 2));
    printf("Value: %d   Address: %p\n", *(branch[1] + 3), (void*)(branch[1] + 3));

total2 = *(branch[1] + 0) + *(branch[1] + 1) + *(branch[1] + 2) + *(branch[1] + 3);
    printf("Total of Branch 2: %d\n", total2);

    return 0;
}
```

```
Branch 1:
Value: 15000  Address: 0x7ffca034a17c
Value: 23000  Address: 0x7ffca034a180
Value: 32000  Address: 0x7ffca034a184
Total of Branch 1: 70000

Branch 2:
Value: 12000   Address: 0x7ffca034a160
Value: 21000   Address: 0x7ffca034a164
Value: 27000   Address: 0x7ffca034a168
Value: 35000   Address: 0x7ffca034a16c
Total of Branch 2: 95000
```

Task 4 (Passing an Array to a Function):

Create calculateTotal() that receives an integer pointer and array size. Use pointer notation to calculate and return the total balance. Call this function for both branch account arrays.

```c
#include <stdio.h>
int calculateTotal(int *p, int size){
    int total = 0;
    int i;

    for (i = 0; i < size; i++) {
        total = total + *(p + i);
    }
    return total;
}
int main(){
    int b1[3] = {15000, 23000, 32000};
    int b2[4] = {12000, 21000, 27000, 35000};

    int result1, result2;

    result1 = calculateTotal(b1, 3);
    result2 = calculateTotal(b2, 4);

    printf("Total of Branch 1: %d\n", result1);
    printf("Total of Branch 2: %d\n", result2);

    return 0;
}
```

```
Total of Branch 1: 70000
Total of Branch 2: 95000
```

Task 5 (Function Pointers):

Create deposit() and withdraw() functions, each accepting two integers and returning an integer result. Use a function pointer to call both operations and display the updated balance.

```c
#include <stdio.h>
int deposit(int balance, int amount){
    return balance + amount;
}
int withdraw(int balance, int amount){
    return balance - amount;
}
int main(){
    int balance = 10000;
    int (*operation)(int, int);

    operation = deposit;
    balance = operation(balance, 5000);
    printf("After deposit of 5000: %d\n", balance);

    operation = withdraw;
    balance = operation(balance, 3000);
    printf("After withdraw of 3000: %d\n", balance);

    return 0;
}
```

```
After deposit of 5000: 15000
After withdraw of 3000: 12000
```

Task 6 (Passing Function Pointers as Arguments):

Create processTransaction() that receives two integers and a function pointer. Use the received pointer to call either deposit() or withdraw() and return the result. Call this function from main() for both operations.

```c
#include <stdio.h>
int deposit(int balance, int amount){
    return balance + amount;
}
int withdraw(int balance, int amount){
    return balance - amount;
}
int processTransaction(int balance, int amount, int (*operation)(int, int)){
    int result;
    result = operation(balance, amount);
    return result;
}
int main(){
    int balance = 10000;

    balance = processTransaction(balance, 5000, deposit);
    printf("After deposit of 5000: %d\n", balance);

    balance = processTransaction(balance, 3000, withdraw);
    printf("After withdraw of 3000: %d\n", balance);

    return 0;
}
```

```
After deposit of 5000: 15000
After withdraw of 3000: 12000
```

IMPORTANT INSTRUCTIONS

Read the complete scenario carefully before starting the programming tasks.

First, perform a complete dry run on paper. Identify the variables, arrays, pointers, functions, and flow of data. Understand how pointers store addresses, how pointer arithmetic accesses array elements and rows, and how function pointers execute banking operations.

After completing the dry run and understanding the logic, implement all six tasks in C.

Remember, the purpose of this assignment is to understand how pointers work in a practical situation. A proper dry run before coding will help you visualize memory addresses, understand program flow, and implement the tasks with confidence.